



Course details

Textbook: Compilers Principles, Techniques, and Tools by *Aho, Sethi, Ullman*

Addl. reference (?): Modern Compiler Implementation in C by *Andrew Appel*

Project: Weightage - 30%

Definition: program that takes a program in *source* language as input and translates it into a program in a *target* language

Motivation:

1. get a better understanding of how programmes work
⇒ write better programs, debugging easier
2. general principles applicable to many tasks, e.g.
HTML-to-text, text formatting, syntax-directed editing, static checkers, shells, etc.

Subtasks: analysis + synthesis

Lexical analysis → Parsing → Semantic analysis →

Intermediate code generation → Optimization →

Code generation

+ Symbol table mgmt., Error handling



Subtasks: front end

Lexical analysis: break up input file into individual tokens

EXAMPLE: `a = b + c`

Syntax analysis / Parsing:

- group tokens into progressively higher-level structures
- structure is defined by rules (CFG)
- structure can be represented using a parse tree / syntax tree

Semantic analysis: type checking, etc.

Intermediate code generation:

- relatively simple to generate from high-level code
- easy to translate into machine code

Code optimization: make code faster/smaller

Code generation:

- generate machine/assembly code
- storage issues have to be addressed (assignment of variables to registers, memory locations, etc.)

Symbol table:

- stores names of identifiers and their attributes (storage, scope, type, number and type of arguments for functions)
- created in the initial stages, used later (type checking / conversion, storage assignment, etc.)

Error handling: parsing errors, semantic errors, etc.

- Preprocessor: handles macros, definitions, file inclusion
- Assembler: converts human readable machine code to binary machine code
 - determine addresses for variables
 - generate code with appropriate addresses
- Linker/loader: resolve references (libraries, .o files) and produce relocatable machine code